Database Queries: Online Retail Platform

1-Find all products in a specific (e.g. ,Electronic) category, displaying their title and price, sorted by price in descending order.

2-Calculate the average rating for each product, including only products with at least one review, and sort by average rating in descending order.

3-List all users who have applied discounts, showing their username and the number of discounts used, sorted by the number of discounts in ascending order.(Bonus)

4-Find products with a specific attribute value (e.g., Color = 'Black') , have been ordered at least twice ,and calculate the total stock quantity for those products.

5-Find all products that have been both ordered and reviewed by the same user. Display the product title, username, rating, and order date.

6-List all products with a stock quantity below the average stock quantity across all products in the Inventory table. Display the product title, stock quantity, and price, sorted by stock quantity ascending.

7-Create a trigger that updates the Inventory stock_quantity when a new order item is inserted into the Order_Items table. Decrease the stock by the ordered quantity.

8-For each category, calculate the total revenue (sum of unit_price * quantity from Order_Items) from orders with status 'Delivered'. Display the category name, total revenue, and number of products sold, sorted by revenue in descending order. Include only categories with sales.

9-Identify users who have spent more than the average total amount spent per user (across all users with orders). Display the username, total spent, and number of orders.

10-Find products that have specific attribute combinations (e.g., Color = 'Black' and Weight = '1KG') , and have been ordered at least three times with a total ordered quantity greater than 5. Display the product title, total ordered quantity, average rating, and the number of categories the product belongs to .Include only products with at least one review.

11-For each user with a premium account, calculate the total number of items in their cart and the total value of those items (based on product prices). Display the username, item count, and total value. (Include only users with items in their cart)

12-Find products that have more reviews than the average number of reviews per product. Display the product title, number of reviews, and current stock quantity.

13-Create a trigger that logs a discount usage in the Discount_Usage table whenever an order is inserted with a non-null discount_id. The trigger should record the user_id, discount_id, and usage_date.(Bonus)

14-For each user who has placed at least one order, calculate the total number of items ordered (sum of quantities from Order_Items) and the total amount spent (sum of total_amount from Orders). Display the username, total items, and total spent.

15-List all products with a price higher than the average price of all products in specific category(e.g. Electronic). Display the product title, price, and stock quantity, sorted by price in descending order.

16-Identify users whose total cart value (quantity * price of items in Cart_Items) exceeds the average total order amount across all orders. Display the username, total cart value, and number of items in the cart, sorted by total cart value in descending order.

17-For each user who has a default address, calculate the total number of orders delivered to that address and the total amount spent on those orders. Display the username, address ID, order count, and total spent.

18-Find products that belong to at least two categories and have an average review rating of 4 or higher. Display the product title, number of categories, and average rating, sorted by average rating in descending order.

19-Create a trigger that logs a record in a new 'Premium_Discount_Usage' table whenever a premium-only discount is applied to an order. The trigger should record the user ID, discount ID, order ID, and usage date.(Bonus)

20-Create a trigger that logs any price changes to products in a new 'Price_Change_Log' table whenever the price column in the Products table is updated. The log should include the product ID, old price, new price, and change date.

21-Create a trigger that automatically assigns a user's first address as the address_id in the Orders table when a new order is inserted, if no address_id is provided. This ensures every order has a valid address.

22-Find products that have a specific EAV attribute value (e.g., Color = 'Black') and have been added to at least three different users' carts with a total cart quantity greater than the average cart quantity across all products. Display the product title, attribute value, total cart quantity, stock quantity, and the number of distinct users who added the product to their carts, sorted by total cart quantity in descending order.

23-Identify products that have specific EAV attribute combination and have been ordered by at least two premium users with a total order value exceeding the average order value for premium users. Display the product title, total order value, number of premium users who ordered, and stock quantity, sorted by total order value in descending order.

24-Find products with a specific EAV attribute that belong to categories with above-average order quantities. Display the product title, category name, attribute value, total ordered quantity, and number of reviews. Include only products with at least one review.

Good Luck:))